

Computer System Architecture

CSEN B501

Dr. Catherine Elias

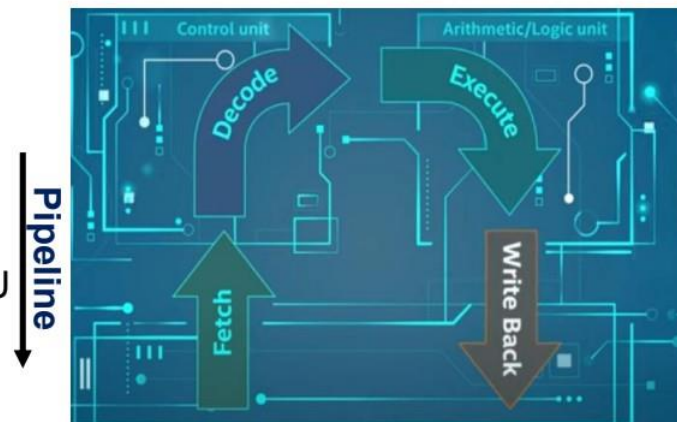
Instruction flow in the CPU

Main structures inside the CPU :

- **Datapath** : Designed Pathways between **Registers**, **ALU** and **control unit**
- **Control unit** : FSMs and control signals controlling **ALU**, **registers** and determining the instruction decoding and the sequence of execution.

Instruction cycle :

1. **(IF)** Instruction Fetch : instruction is fetched using PC,MDR,MAR.
2. **(ID)** Instruction Decode : instruction decoded within the IR
3. **(IE)** Instruction Execute : the decoded instruction is executed in ALU
4. **(WB)** Write Back : The result is written in register/memory



Building Datapath (R-format)

R-Type : **add \$t0, \$s1, \$s2**

It adds the values in registers \$s1 and \$s2 and stores the result in register \$t0

Opcode	rs	rt	rd	shamt	funct
000000	10001	10010	01000	00000	100000

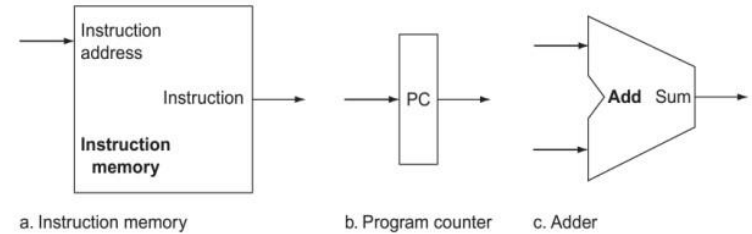
R-Type

6	5	5	5	5	6
opcode	rt	rs	rd	shift	func

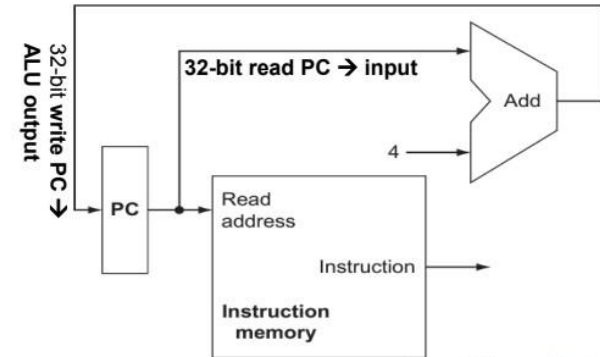
Notes for designing the Datapath in IF phase :

1. Read the 32-bit instruction address from the **PC**
2. Read the address and Copy it to MAR and fetch for Instruction
3. add 4 bytes to the PC (next instruction address)
In the adder and write it to the PC.

State elements needed for IF



A



Building Datapath (R-format)

R-Type : **add \$t0, \$s1, \$s2**

It adds the values in registers \$s1 and \$s2 and stores the result in register \$t0

Opcode **rs** **rt** **rd** **shamt** **funct**
 000000 10001 10010 01000 00000 100000

Notes for designing the Datapath :

1. There are 32 General-purpose registers → **5-bits** wire/bus needed to represent each **register**.
2. Each register is 32-bit wide → **32-bits** wire/bus needed to read/write the register **Data**
3. In **add \$t0, \$s1, \$s2** we need to read 32-bit data from two registers **\$s1** , **\$s2** and supply the data to the Alu then save the result in **\$t0** .

R-Type

6	5	5	5	5	6
opcode	rt	rs	rd	shift	func

Designing the width of the pathways between register file , Alu and control unit

Reg name Reg address

•	\$zero	00000
•	\$at	00001
•	\$v0	00010
•	\$v1	00011
•	\$a0	00100
•	...a...	
•	\$a3	00111
•	\$t0	01000
•	t...	
•	\$t7	01111
•	\$s0	10000
•	...s.....	
•	\$s7	10111
•	\$t8	11000
•	\$t9	11001
•	\$k0	11010
•	\$k1	11011
•	\$gp	11100
•	\$sp	11101
•	\$fp	11110
•	\$ra	11111

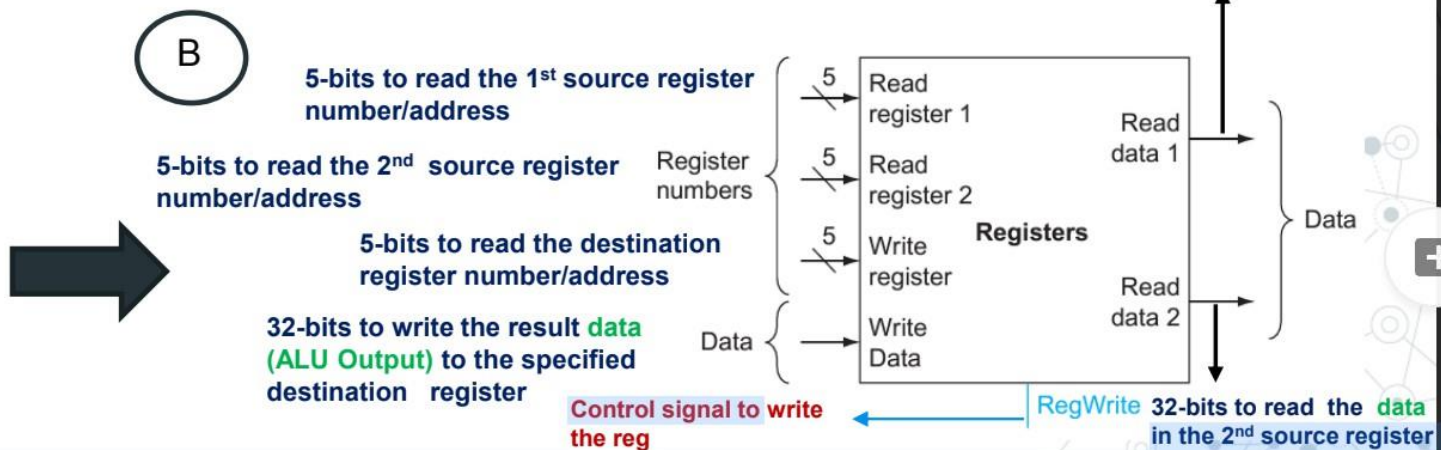
Building Datapath (R-format)

Notes for designing the Datapath in ID phase :

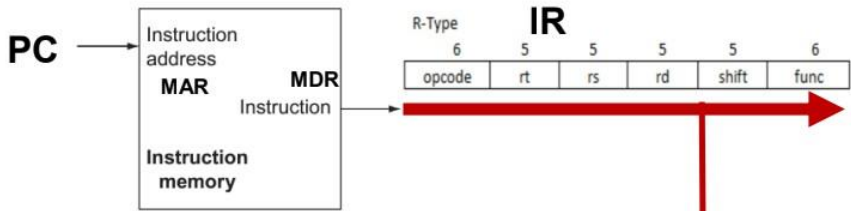
1. There are 32 General-purpose registers → **5-bits** wire/bus needed to represent each **register**
2. Each register is 32-bit wide → **32-bits** wire/bus needed to read/write the register **Data**
3. In **add \$t0, \$s1, \$s2** we need to read 32-bit data from two registers and supply the data to the Alu .

32-bits to read the data in the 1st source register → ALU input

Designing the width and the number of the pathways between register file , ALU and control unit



Building Datapath (R-format)



a. Instruction memory

Combining A&B Diagrams to generalize a Datapath for R-format

State elements needed for R-format :

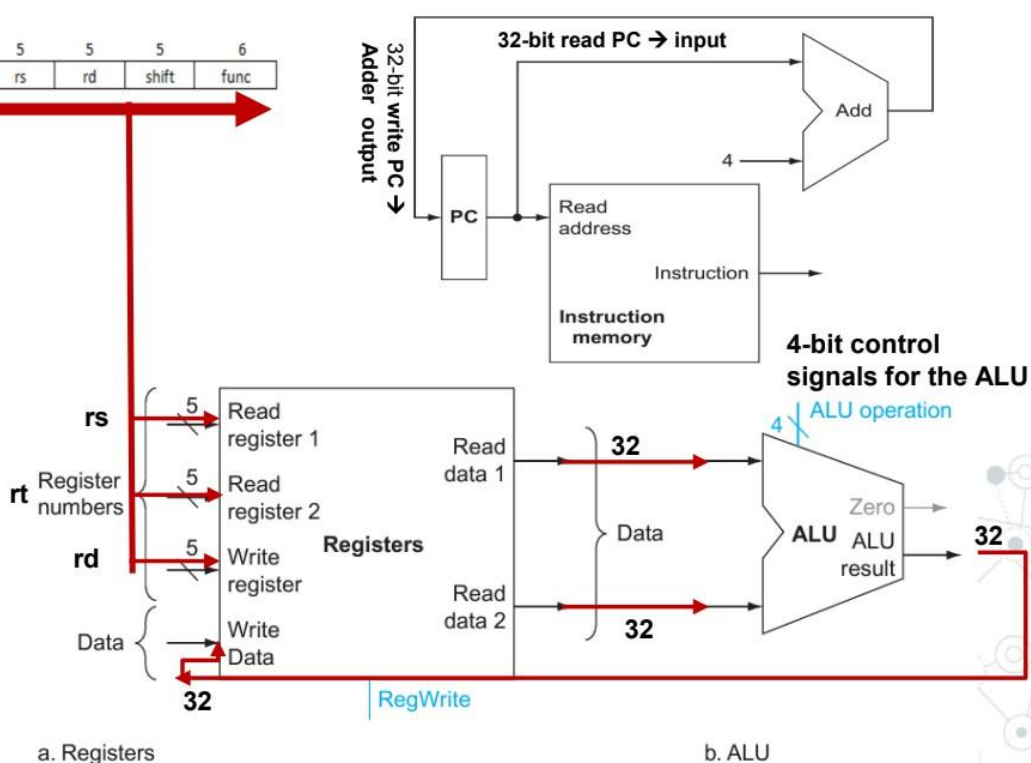
PC

Instruction memory

Adder

Register-File

ALU



a. Registers

b. ALU

Building Datapath (I-format)

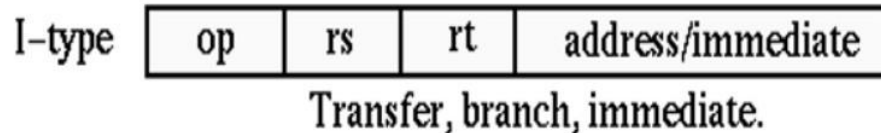
I-Format : lw \$t1,100(\$t2)

6	5	5	16
opcode	rs	rt	immediate
100011	01010	01001	0000000001100100
lw	\$t2	\$t1	100

Notes for designing the Datapath :

1. **\$t1**: The destination register where the loaded value will be stored. In binary, **\$t1** is represented as **01001** → **5-bit**
2. **100**: The immediate offset value. It's a signed 16-bit value in two's complement representation. For 100, the binary representation is **0000000001100100** → **16-bit**
3. **(\$t2)**: The base register holding the base address **32 bit**. In binary, **\$t2** is represented as **01010** → **5-bits**

I-instructions compute a memory address by adding the base register, which is \$t2, to the 16-bit signed off set field contained in the instruction.



Building Datapath (I-format)

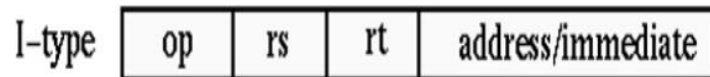
I-Format : lw \$t1,100(\$t2)

6	5	5	16
opcode	rs	rt	immediate
100011	01010	01001	0000000001100100
lw	\$t2	\$t1	100

4. I-instructions compute a memory address by adding the base register, which is \$t2, to the 16-bit signed off set field contained in the instruction.

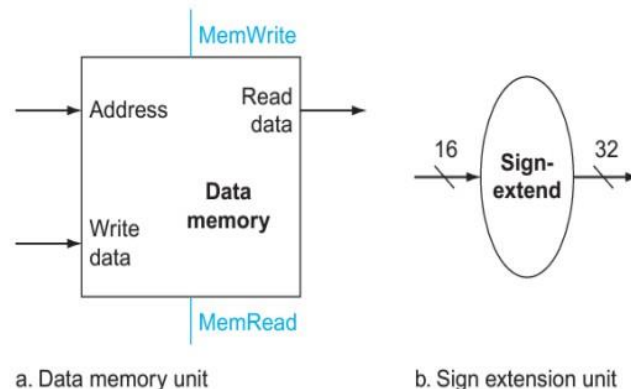
5. we will need a unit to sign-extend the 16-bit off set field in the instruction to a 32-bit signed value to be compatible with the 32-bit operations.

6. a data memory unit to read **from** (load) or write (store) to hence, data memory has read and write control signals, an address input (32) , and an input for the data (32) to be written into memory.



Transfer, branch, immediate.

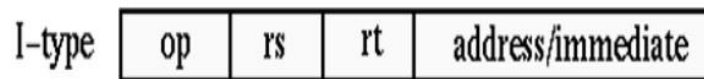
Data memory and sign-extend Units are needed for Load and stores



Building Datapath (I-format)

I-Format : `lw $t1,100($t2)`

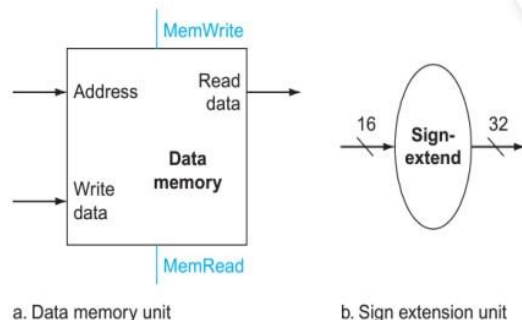
6	5	5	16
opcode	rs	rt	immediate
100011	01010	01001	0000000001100100
<code>lw</code>	<code>\$t2</code>	<code>\$t1</code>	<code>100</code>



Transfer, branch, immediate.

Data memory and sign-extend Units are needed for Load and stores.

- a data memory unit to read from (**load**) or write (**store**) to hence, data memory has read and write control signals, an address input (32) , and an input for the data (32) to be written into memory.
- The offset value is a signed value (it can be negative or positive) .
- The offset value is extended to 32 bits and **added directly** without any shifting.
- The offset value in **lw** / **sw** / **addi** is specified in Bytes so it does not need to be shifted.



a. Data memory unit

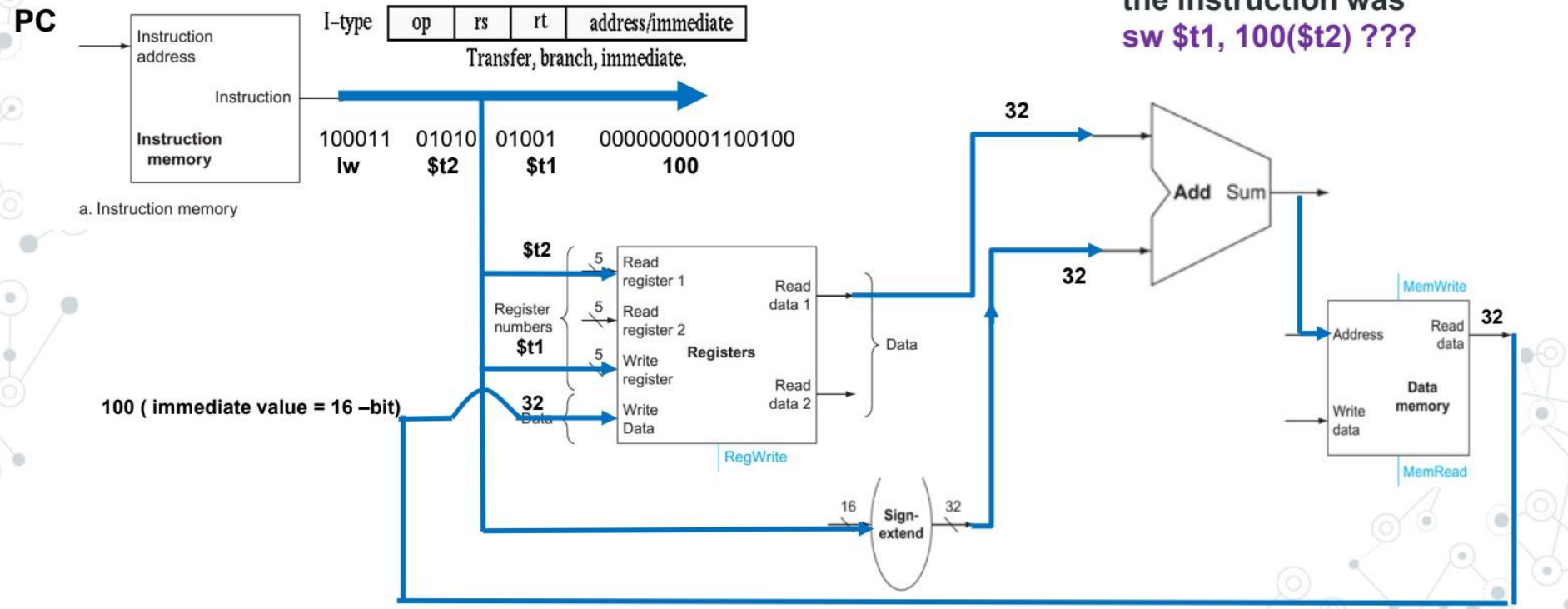
b. Sign extension unit

Building Datapath (I-format)

I-Format : lw \$t1,100(\$t2)

IR

What will be differ if the instruction was **sw \$t1, 100(\$t2) ???**

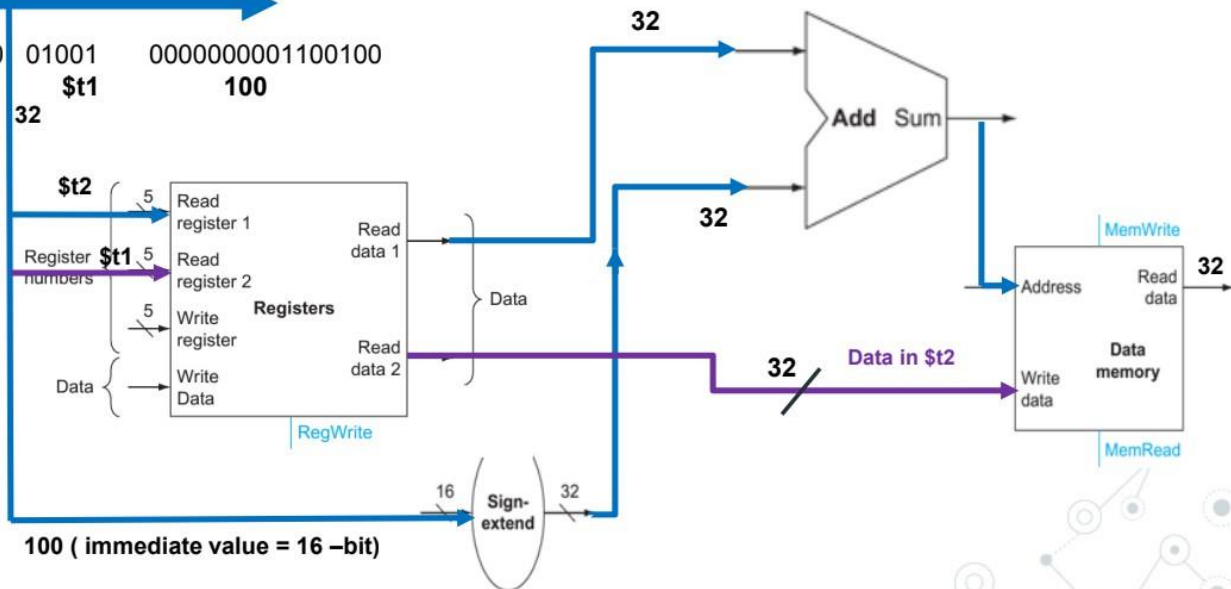


I-Format : sw \$t1,100(\$t2)

IR



1. **Read both \$t1 and \$t2**
2. The base address is stored in \$t2 so it is added with the byte offset (100) to calculate the target memory address ($100 + \$t2$)
3. **Write the data in \$t1 to the data memory in the specified address.**



Building Datapath (I-format)

I-Format : beq \$t1,\$t2,100

opcode	rs	rt	offset
000100	01001	01010	0000000001100100

Notes for designing the Datapath :

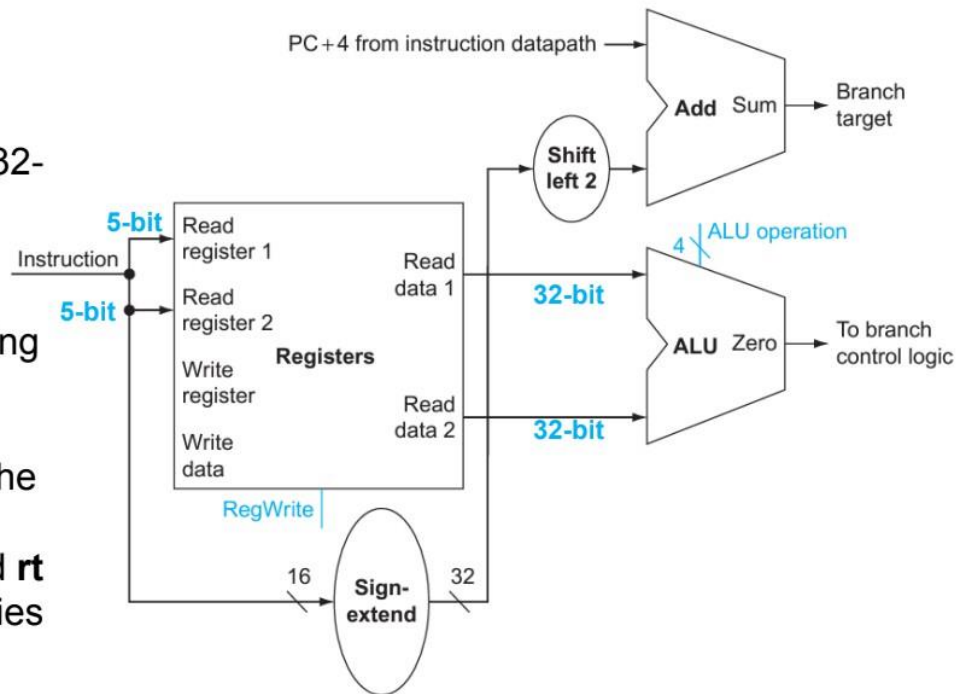
1. The **beq** instruction has three operands, two registers that are compared for equality, and a 16-bit off set used to compute the branch target address relative to the branch instruction address.
2. To implement this instruction, we must compute the branch target address by adding the sign-extended off set field of the instruction to the PC.
3. The instruction set architecture specifies that the base for the branch address calculation is the address of the instruction following the branch. Since we compute **PC + 4** (the address of the next instruction) in the instruction fetch Datapath
4. The architecture also states that the **offset field is shifted left 2 bits** so that it is a word offset (**it is given in words not in bytes**) . this shift increases the effective range of the off set field by a factor of 4.

Building Datapath (I-format)

I-Format : beq \$t1,\$t2,100

opcode	rs	rt	offset
000100	01001	01010	0000000001100100
beq	\$t1	\$t2	100

- The sign-extend unit extends the 16-bit offset to 32-bit number so it can be compatible among the Adder inputs .
- The **16-bit offset** from the instruction format is extended then **shifted left by 2** through the shifting unit.
- The output of the shifting unit is added with the address of the next instruction (**PC+4**) to output the targeted branch address .
- The ALU inputs are the 32-bit of data from **rs** and **rt** and the output is the comparing result that specifies whether to branch or not. (**Logic**)



Building Datapath (I-format)

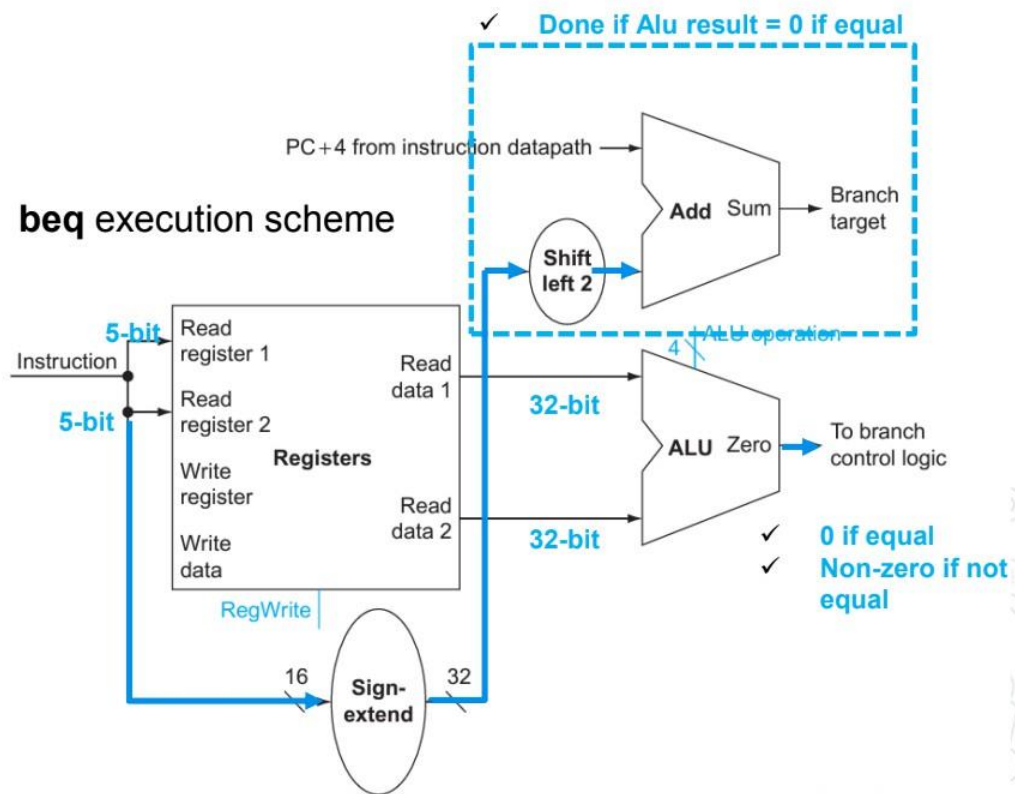
I-Format : beq \$t1,\$t2,100

opcode	rs	rt	offset
000100	01001	01010	0000000001100100
beq	\$t1	\$t2	100

State elements needed for I-format :

- ✓ **PC**
- ✓ **Instruction memory**
- ✓ **Adder**
- ✓ **Register-File**
- ✓ **ALU**
- ✓ **Shifting unit**
- ✓ **Sign-extend**

beq execution scheme



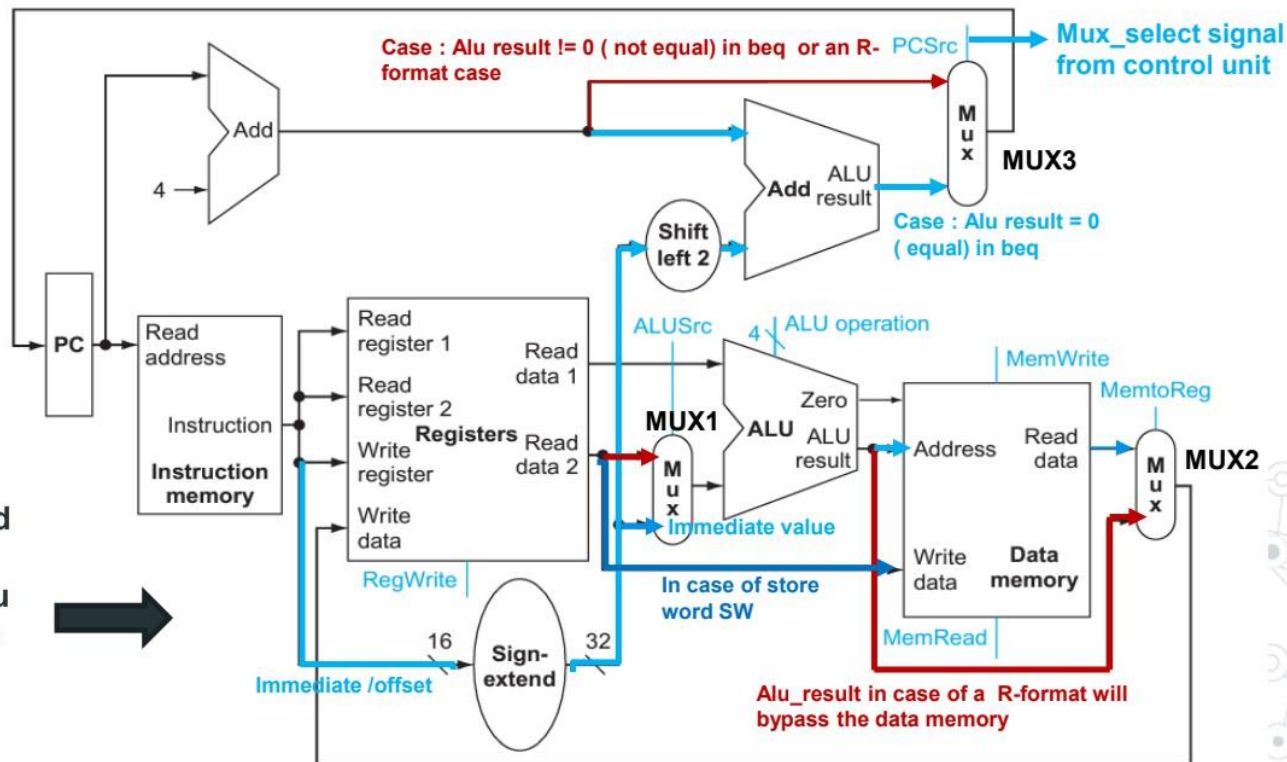
Generalized Datapath for R & I formats

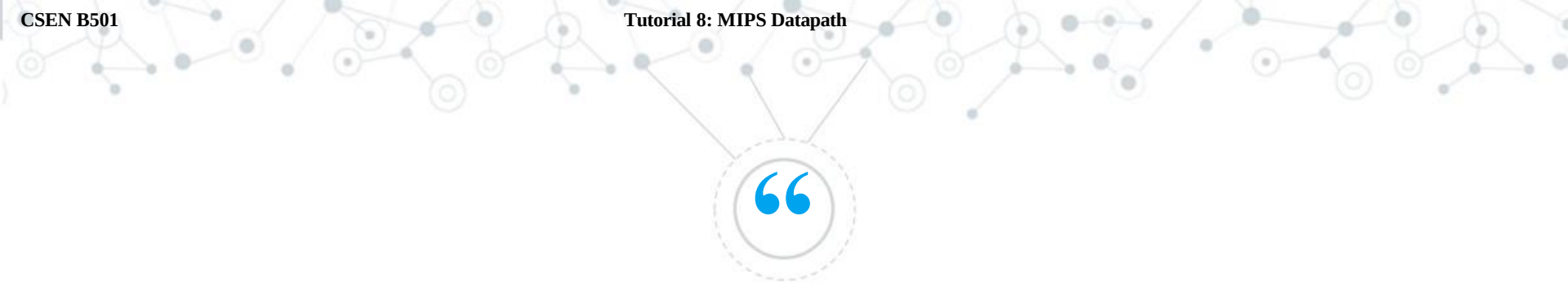
This simple implementation covers

- ✓ load word (lw)
- ✓ store word (sw)
- ✓ branch equal (beq)
- ✓ add, sub, AND, OR, and set on less than.

- Common Datapath
- I-Format Datapath
- R-Format

By augmenting both R & I diagrams and using 3 MUXs to represent different possible paths. The mux select and Alu signals are initiated by the control unit based on opcode, func bits.





“

THANK YOU